Design and Implementation of Dual-Port SRAM in 90nm CMOS Technology Using Cadence

Chapter 1: Introduction

1.1 Background

Static Random Access Memory (SRAM) plays a vital role in modern VLSI systems, serving as cache memory, register files, and buffers in high-performance processors. Unlike DRAM, SRAM does not require periodic refreshing, which makes it faster and more reliable for applications demanding low latency.

Dual-port SRAM extends the conventional architecture by allowing simultaneous access through two independent ports, enabling parallel read/write operations. This capability is essential in multi-core processors, DSPs, and networking systems where high throughput and concurrent data access are critical.

1.2 Motivation

As technology scales, efficient memory design becomes increasingly challenging. The 90nm CMOS process node offers a balance between speed, power, and area, making it suitable for academic and industrial research. Implementing a dual-port SRAM using the complete RTL-to-GDSII flow provides practical exposure to industry-standard design methodologies and highlights challenges in synthesis, placement, routing, and verification.

1.3 Objectives

The objectives of this project are:

- To design a dual-port SRAM architecture at the RTL level.
- To implement the design using Cadence tools in 90nm CMOS technology.
- To carry out the full RTL-to-GDSII flow, including synthesis, floorplanning, placement, routing, and sign-off.
- To verify functionality and analyze performance parameters such as area, timing, and power.

1.4 Scope of the Project

This project is limited to the design and physical implementation of dual-port SRAM. The scope includes:

- RTL coding and functional simulation.
- Logic synthesis and optimization.
- Physical design flow up to GDSII generation.
- Verification through DRC, LVS, and timing analysis.
- Performance evaluation based on post-layout results.

1.5 Organization of the Report

- Chapter 1 introduces the project, motivation, objectives, and scope.
- Chapter 2 presents a literature survey of SRAM architectures and related work.
- Chapter 3 explains the research methodology, focusing on the RTL-to-GDSII flow.
- Chapter 4 discusses results and provides analysis of the implemented design.
- Chapter 5 concludes the project with key findings and future scope.

Chapter 2: Literature Survey

2.1 Overview of SRAM

Static Random Access Memory (SRAM) is widely used in digital systems due to its high speed and low power consumption compared to DRAM. Conventional single-port SRAM allows only one access at a time, which limits throughput in applications requiring concurrent operations. Dual-port SRAM addresses this limitation by enabling simultaneous access through two independent ports, making it suitable for multi-core processors, DSPs, and networking systems.

2.2 Single-Port vs. Dual-Port SRAM

- **Single-Port SRAM**: Simple architecture, lower area, and power requirements, but limited to one read/write operation at a time.
- **Dual-Port SRAM**: More complex design, larger area overhead, but supports parallel read/write operations, improving system performance.
- Trade-offs include increased design complexity, higher leakage currents, and challenges in maintaining cell stability.

2.3 Prior Research on Dual-Port SRAM

Several studies have focused on optimizing dual-port SRAM designs:

- Techniques to reduce power consumption through transistor sizing and low-leakage cells.
- Architectural improvements to enhance speed and minimize access conflicts.
- Layout optimization strategies to balance area efficiency with performance.
- Use of advanced CMOS nodes (e.g., 65nm, 45nm) for scaling, though 90nm remains a widely studied node for academic projects due to its balance of complexity and accessibility.

2.4 RTL-to-GDSII Flow in VLSI Design

The RTL-to-GDSII flow is the industry-standard methodology for digital IC design. It involves:

- **RTL Design**: Writing behavioral code (e.g., Verilog/VHDL) to describe functionality.
- **Logic Synthesis**: Converting RTL into gate-level netlist optimized for timing, area, and power.
- **Floorplanning & Placement**: Defining chip layout and placing standard cells.
- **Routing**: Connecting cells with metal layers while meeting design rules.
- **Verification**: Ensuring correctness through Design Rule Check (DRC), Layout Versus Schematic (LVS), and Static Timing Analysis (STA).
- **GDSII Generation**: Producing the final mask data for fabrication.

2.5 Gaps in Existing Literature

- Many prior works emphasize single-port SRAM due to simplicity, leaving dual-port designs less explored.
- Limited studies focus on complete RTL-to-GDSII implementation of dual-port SRAM in 90nm CMOS.
- Challenges such as simultaneous access conflicts, leakage power, and layout optimization remain open areas for research.

2.6 Summary

The literature highlights the importance of dual-port SRAM in high-performance systems and the need for efficient design methodologies. While prior research has addressed power and speed optimization, fewer

studies have demonstrated a full RTL-to-GDSII flow for dual-port SRAM in 90nm CMOS technology. This project aims to bridge that gap by implementing and analyzing such a design using Cadence tools.

Chapter 3: Research Methodology

3.1 Design Flow Overview

The project follows the complete **RTL-to-GDSII flow**, which is the industry-standard methodology for digital IC design. The flow consists of:

- **RTL Design**: Writing Verilog code to describe the dual-port SRAM functionality.
- **Functional Simulation**: Verifying correctness of RTL using testbenches.
- **Logic Synthesis**: Converting RTL into a gate-level netlist optimized for timing, area, and power.
- **Floorplanning**: Defining chip dimensions, power planning, and placement constraints.
- **Placement and Routing**: Placing standard cells and routing interconnections while meeting design rules.
- **Verification**: Performing Design Rule Check (DRC), Layout Versus Schematic (LVS), and Static Timing Analysis (STA).
- **GDSII Generation**: Producing the final layout database for fabrication.

3.2 Proposed Dual-Port SRAM Architecture

The architecture of the dual-port SRAM is designed to allow simultaneous access through two independent ports. Each port has its own address, data input, data output, and control signals. The design includes:

- **Memory Array**: Central block of SRAM cells arranged in rows and columns.
- **Row and Column Decoders**: Independent decoders for each port to select memory cells.
- **Control Logic**: Manages read/write operations and resolves conflicts when both ports access the same location.
- **Data Paths**: Separate input/output paths for each port to enable parallel operations.

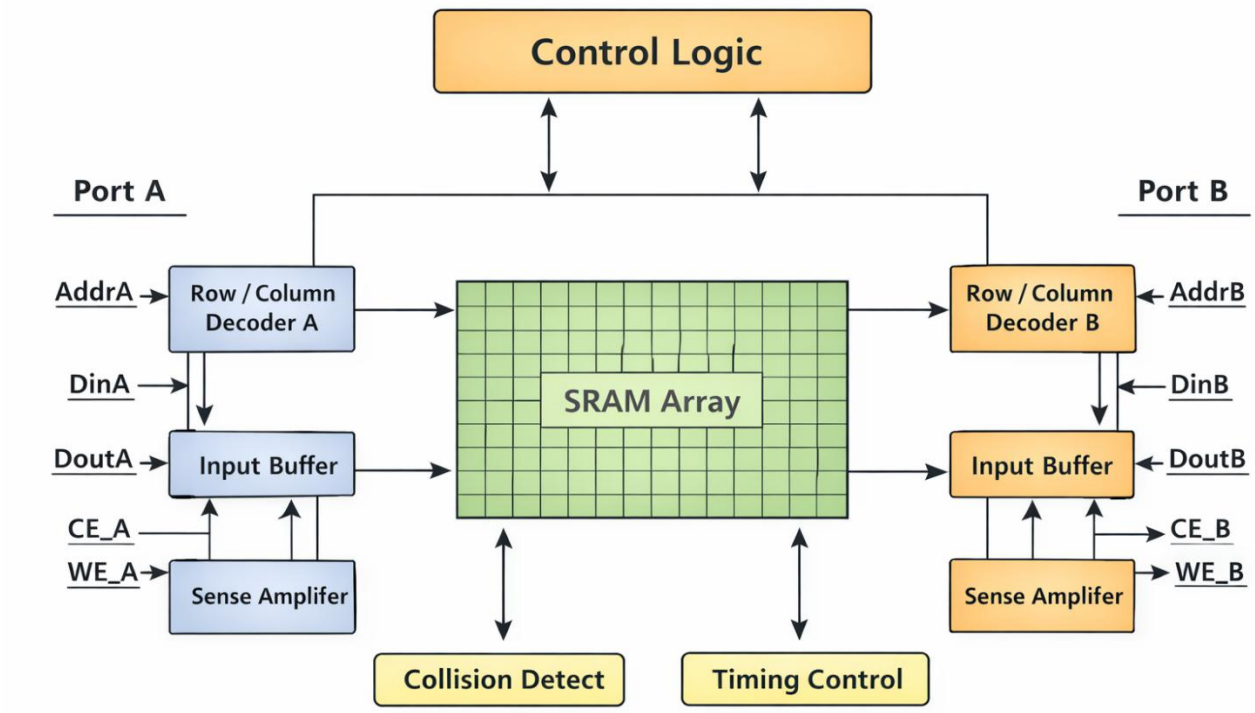

Figure 3.1: Proposed Dual-Port SRAM Block Diagram

3.3 Implementation in Cadence

The design was implemented using Cadence tools in 90nm CMOS technology. The methodology included:

- **Schematic Entry**: Creating the SRAM schematic in Cadence Virtuoso.
- **Simulation**: Running pre-layout simulations to verify functionality.
- **Synthesis and Netlist Generation**: Using Cadence RTL Compiler for synthesis.
- **Physical Design**: Performing floorplanning, placement, and routing in Cadence Innovus.
- **Verification**: Ensuring correctness with DRC, LVS, and STA checks.
- **Final GDSII**: Exporting the verified layout for fabrication readiness.

3.4 Summary

This chapter outlined the methodology adopted for designing and implementing the dual-port SRAM. The proposed architecture was described, and the complete RTL-to-GDSII flow was detailed. The next chapter will present the results obtained from simulation, layout verification, and performance analysis.

Chapter 4: Results and Discussion

4.1 Functional Verification

The dual-port SRAM was verified through simulation waveforms. Each waveform is labelled and discussed:

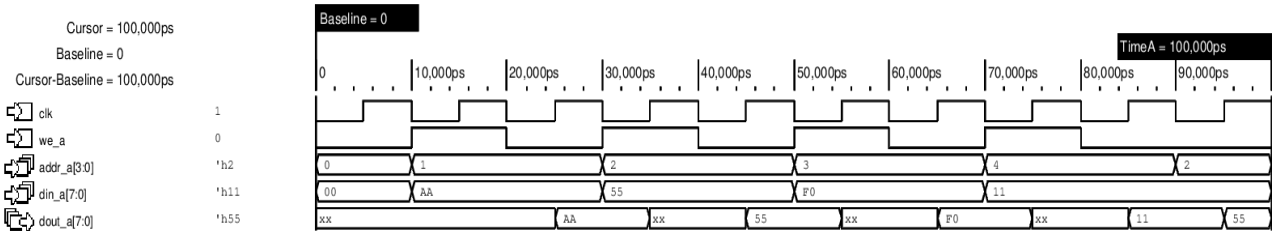

Figure 4.1: Write First Read-Through

Shows correct data retrieval from the memory array when a read operation is initiated.

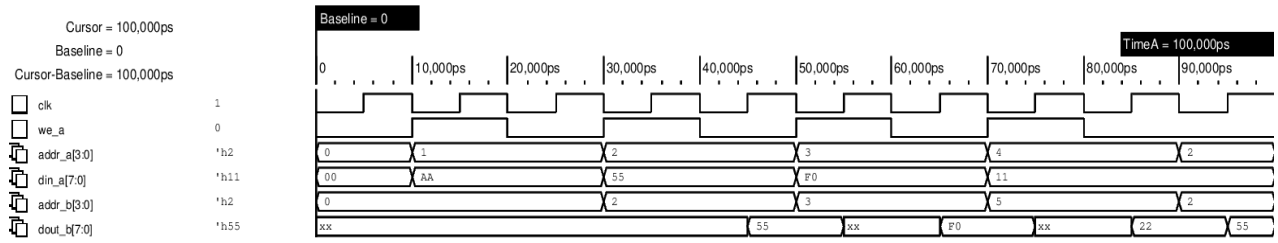

Figure 4.2: Cross-Port Operation

Demonstrates simultaneous access from both ports, validating independent operation.

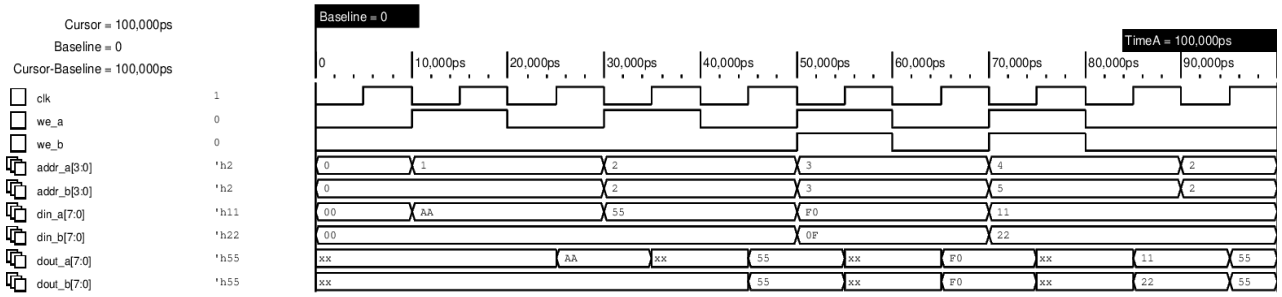

Figure 4.3: Write-Write Collision

Illustrates behavior when both ports attempt to write to the same memory location, highlighting conflict resolution logic.

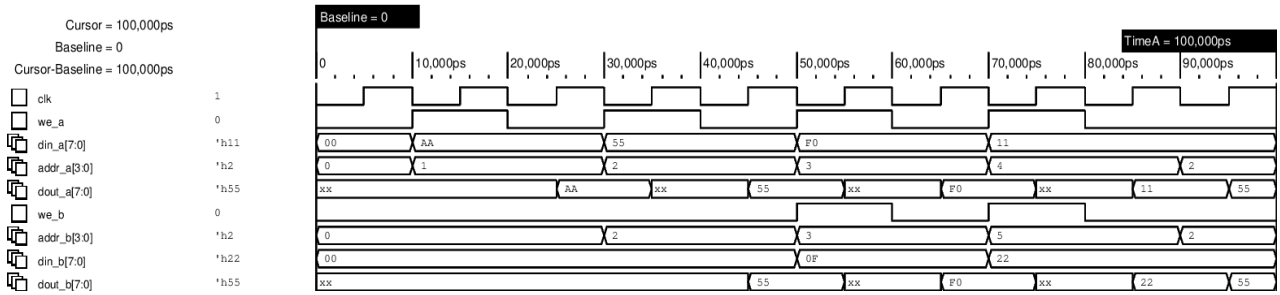

Figure 4.4: Independent Operation

Confirms proper functioning when ports access different addresses concurrently.

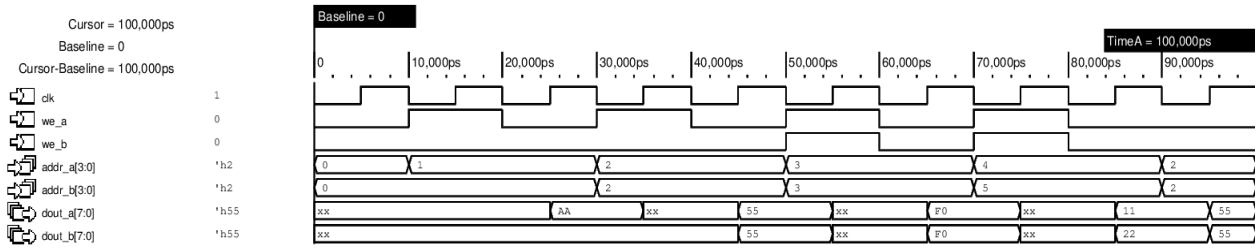

Figure 4.5: Simultaneous Read

Validates parallel read operations from both ports without interference.

4.2 Schematic and Netlist

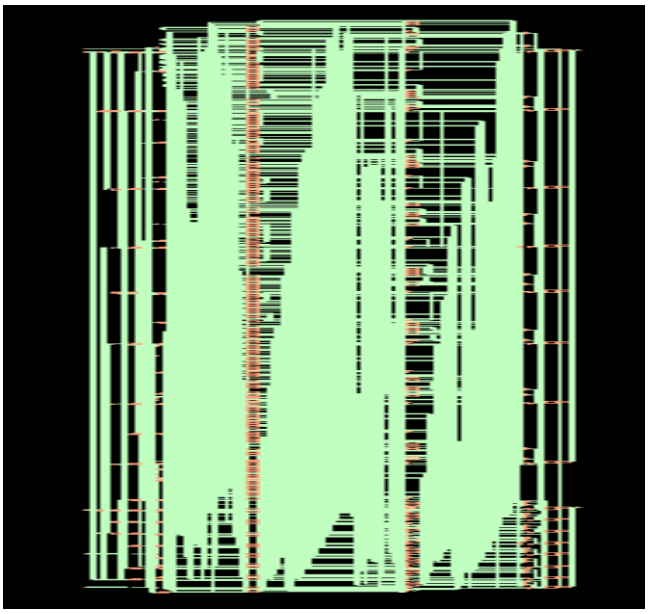

Figure 4.6: SRAM Schematic

Displays the memory array, decoders, and control logic blocks.

- The synthesized netlist confirms correct translation of RTL into gate-level representation.

4.3 Area Report

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Mar 17 2026  02:47:24 pm
  Module:                 sram_dp
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------
sram_dp                  448   7179.953     0.000     7179.953 <none> (D)
  (D) = wireload is default in technology library
```

Figure 4.7: Area Report Output

Shows core area utilization, cell density, and routing congestion.

4.4 Power Report

```
Instance: /sram_dp
Power Unit: W
PDB Frames: /stim#0/frame#0
  ----------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ----------------------------------------------------------------------
       memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     register     3.63559e-05  4.21904e-04  1.30590e-05  4.71318e-04  86.25%
        latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        logic     7.98062e-06  1.71023e-05  1.26479e-05  3.77308e-05   6.90%
         bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        clock     0.00000e+00  0.00000e+00  3.73896e-05  3.73896e-05   6.84%
          pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
           pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ----------------------------------------------------------------------
    Subtotal     4.43365e-05  4.39006e-04  6.30965e-05  5.46439e-04  99.99%
  Percentage           8.11%       80.34%       11.55%      100.00% 100.00%
  ----------------------------------------------------------------------
```

Figure 4.8: Power Report Output

Provides dynamic power, leakage power, and total consumption values.

4.5 Timing Report

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Mar 17 2026  02:47:09 pm
  Module:                 sram_dp
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================


Path 1: MET (5995 ps) Setup Check with Pin mem_reg[15][7]/CK->SE
          Group: clk
     Startpoint: (R) addr_a[2]
          Clock: (R) clk
       Endpoint: (R) mem_reg[15][7]/SE
          Clock: (R) clk

                    Capture       Launch
        Clock Edge:+   10000            0
         Drv Adjust:+      0            0
       Src Latency:+       0            0
       Net Latency:+       0 (I)        0 (I)
           Arrival:=   10000            0

             Setup:-     379
     Required Time:=    9621
      Launch Clock:-       0
       Input Delay:-    2000
         Data Path:-    1625
             Slack:=    5995

Exceptions/Constraints:
  input_delay             2000            dpsram.sdc_line_11_3_1

#-------------------------------------------------------------------------------------
#   Timing Point    Flags   Arc   Edge   Cell     Fanout  Load Trans Delay Arrival Instance
#                                                        (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------
  addr_a[2]         -       -     R     (arrival)      1   1.7     0     0    2000    (-,-)
  drc_buf_sp16122/Y -       A->Y  R     BUFX2         36 167.5   711   562    2562    (-,-)
  g10461__2346/Y    -       B->Y  F     NAND2XL        4  10.0   384   343    2905    (-,-)
  g10443__5122/Y    -       A->Y  R     NOR2XL         8  30.4   899   720    3625    (-,-)
  mem_reg[15][7]/SE <<<     -     R     SDFFQX1        8     -     -     0    3625    (-,-)
#-------------------------------------------------------------------------------------
```

Figure 4.9: Timing Report Output

Displays setup time, hold time, Worst Negative Slack (WNS), and Total Negative Slack (TNS).

4.6 Clock Tree Debugger

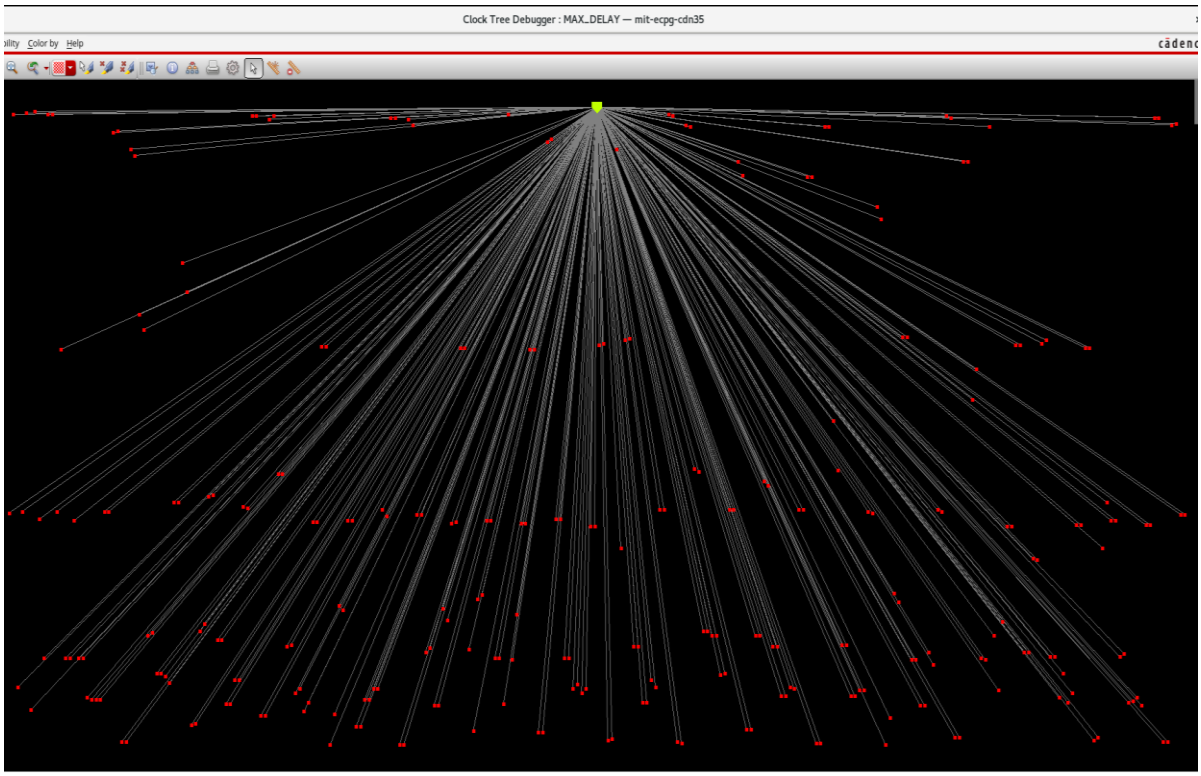

Figure 4.10: Clock Tree Debugger Output

Illustrates clock distribution, skew minimization, and insertion delay optimization.

4.7 Timing Debug (Setup and Hold)

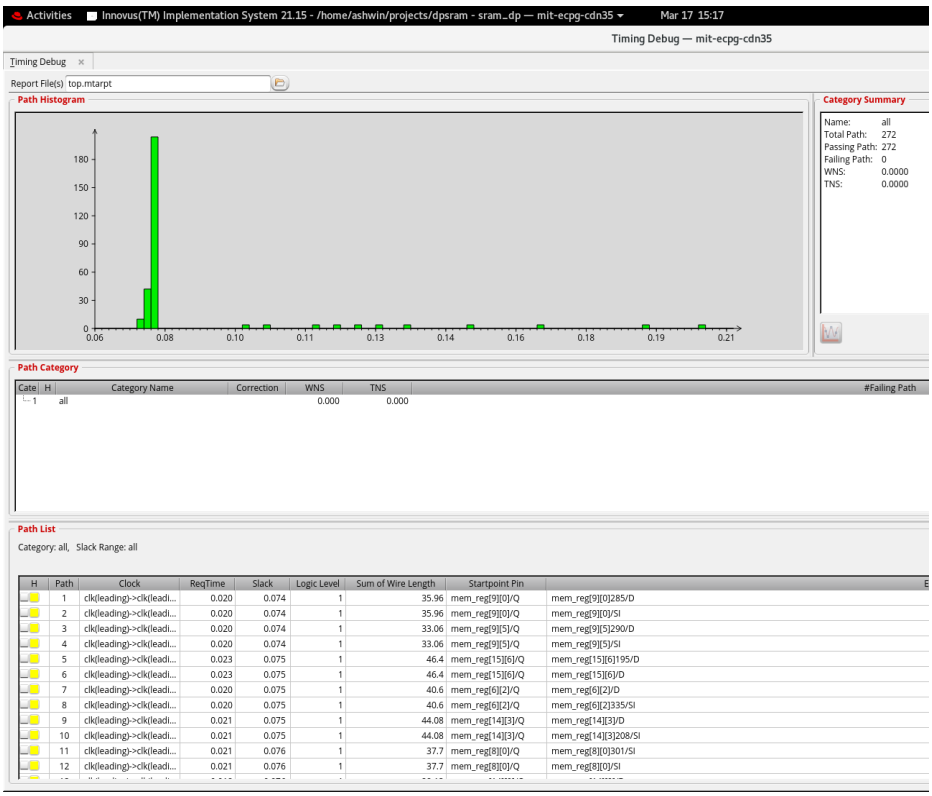

Figure 4.11: Timing Debug Output

Confirms setup and hold constraints are satisfied across critical paths.

4.8 Final Layout

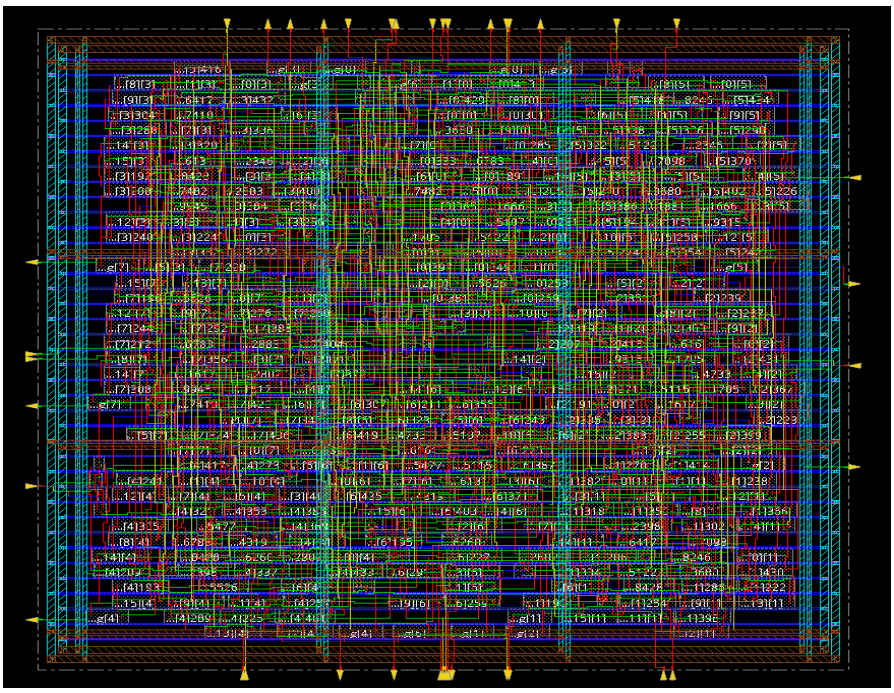

Figure 4.12: 2D Layout View

Shows placement and routing of standard cells.

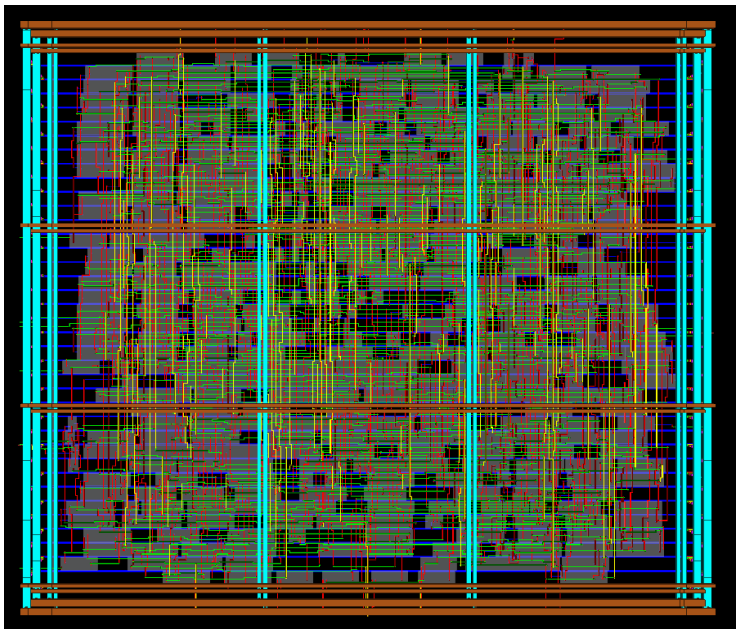

Figure 4.13: 3D Layout View

Visualizes interconnect layers and metal stack.

Figure 4.14: GDSII File Snapshot

Represents the final mask data ready for fabrication.

4.9 Discussion

The results confirm that the dual-port SRAM design meets functional and physical requirements. Waveforms validated correct operation, while area, power, and timing reports demonstrated efficiency in 90nm CMOS technology. The final GDSII layout ensures fabrication readiness, completing the RTL-to-GDSII flow successfully.

Chapter 5: Conclusion

5.1 Summary of Work

This project successfully designed and implemented a **Dual-Port SRAM** using the **RTL-to-GDSII flow** in **90nm CMOS technology** with Cadence tools. The design covered RTL coding, synthesis, placement, routing, and final GDSII generation. Functional verification through waveforms confirmed correct operation under multiple scenarios, while physical design reports validated area, power, and timing efficiency.

5.2 Key Achievements

- Designed and verified a dual-port SRAM architecture supporting parallel operations.
- Completed full RTL-to-GDSII flow using Cadence tools.
- Validated functionality through waveform simulations.
- Achieved timing closure with setup and hold constraints satisfied.
- Generated final GDSII layout, ready for fabrication.

5.3 Limitations

- Implementation restricted to 90nm node; scaling to advanced nodes may introduce new challenges.
- Limited exploration of advanced low-power techniques.
- Focused primarily on functional correctness and physical implementation.

5.4 Future Scope

- Extend design to smaller technology nodes (65nm, 45nm, FinFET).
- Incorporate low-power techniques such as clock gating and multi-Vt cells.
- Explore error detection and correction mechanisms for reliability.
- Integrate SRAM into larger SoC designs for real-world evaluation.
- Automate design space exploration for optimized trade-offs.

References

1. Weste, N. H. E., & Harris, D. M. *CMOS VLSI Design: A Circuits and Systems Perspective*. Pearson Education, 4th Edition.
2. Rabaey, J. M., Chandrakasan, A., & Nikolic, B. *Digital Integrated Circuits: A Design Perspective*. Prentice Hall, 2nd Edition.
3. Cadence Design Systems, *Virtuoso Layout Suite User Guide*.
4. Cadence Design Systems, *Innovus Implementation System Documentation*.
5. Kang, S. M., & Leblebici, Y. *CMOS Digital Integrated Circuits: Analysis and Design*. McGraw-Hill.
6. IEEE Xplore Digital Library – Research papers on dual-port SRAM architectures in 90nm CMOS technology.
7. ACM Digital Library – Studies on RTL-to-GDSII implementation methodologies.

Thank You for Reading!
Let's Connect
If you're passionate about Physical Design, RTL, or the Cadence Flow, I'd love to chat.
If you found this report useful, please Like and Repost the original post. It helps more engineers find this resource!
More projects coming soon... Stay tuned!